Check for updates

Received: 29 September 2022 | Revised: 4 February 2023 | Accepted: 7 March 2023

DOI: 10.1002/pmic.202200041

RESEARCH ARTICLE

Proteomics
Proteomics and Systems Biology

A transformer architecture for retention time prediction in liquid chromatography mass spectrometry-based proteomics

Thang V. Pham[1,2] | Vinh V. Nguyen[3] | Duong Vu[4] | Alex A. Henneman[1,2] | Robin A. Richardson[5] | Sander R. Piersma[1,2] | Connie R. Jimenez[1,2]

[1]Amsterdam UMC, location Vrije Universiteit Amsterdam, OncoProteomics Laboratory, Medical Oncology, Amsterdam, The Netherlands

[2]Cancer Center Amsterdam, Cancer Biology and Immunology, Amsterdam, The Netherlands

[3]University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam

[4]Westerdijk Fungal Biodiversity Institute, Uppsalalaan 8, Utrecht, The Netherlands

[5]Netherlands eScience Center, The Netherlands

Correspondence
Thang V. Pham and Connie R. Jimenez, Amsterdam UMC, location Vrije Universiteit Amsterdam, OncoProteomics Laboratory, Medical Oncology, De Boelelaan 1117, Amsterdam, The Netherlands.
Email: t.pham@amsterdamumc.nl and c.jimenez@amsterdamumc.nl

Abstract

Accurate retention time (RT) prediction is important for spectral library-based analysis in data-independent acquisition mass spectrometry-based proteomics. The deep learning approach has demonstrated superior performance over traditional machine learning methods for this purpose. The transformer architecture is a recent development in deep learning that delivers state-of-the-art performance in many fields such as natural language processing, computer vision, and biology. We assess the performance of the transformer architecture for RT prediction using datasets from five deep learning models Prosit, DeepDIA, AutoRT, DeepPhospho, and AlphaPeptDeep. The experimental results on holdout datasets and independent datasets exhibit state-of-the-art performance of the transformer architecture. The software and evaluation datasets are publicly available for future development in the field.

KEYWORDS
deep learning, DIA-MS, retention time prediction, spectral library, transformer architecture

1 | INTRODUCTION

The retention time (RT) of a peptide refers to the time point it eludes from a liquid chromatography (LC) column in a proteomics experiment using a mass spectrometer (MS) coupled with LC separation. The peptide RT depends largely on the intrinsic properties of its amino acid composition. Accurate RT prediction has several applications. It is an integral part of a spectral library, a collection of mass spectra and RTs, for analysis of data-independent acquisition mass spectrometry (DIA-MS) data [1–3]. RT prediction can also be used to improve peptide identification confidence to control false discovery rate [4] and to identify chimeric fragmentation spectra [5]. Previous works have demonstrated the superiority of the deep learning approach over traditional machine learning methods for RT prediction [4, 6–9]. In this paper, we examine the performance of a new deep learning architecture called the transformer [10] that has recently revolutionized the field of artificial intelligence.

The long short-term memory (LSTM) architecture [11], a workhorse model in natural language processing (NLP), has been used extensively for RT prediction. This is because there is a parallel between variable-length sequences of peptides to words and sentences in NLP whereby amino acids correspond to words in a variable-length sentence. The transformer model is a recent breakthrough in NLP which enables parallel processing with high accuracy [10, 12] and is gradually replacing LSTM. The transformer architecture models long range dependency via a multi-layer self-attention mechanism. Next to successful applications in NLP, it has demonstrated state-of-the-art performance in other fields such as computer vision [13] and biology [14]. A transformer model is used for prediction of MS/MS spectrum intensity in

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.
© 2023 The Authors. Proteomics published by Wiley-VCH GmbH.

proteomics [15]. Lou et al. [16] use a transformer block after an LSTM block for RT prediction. However, there is no comparison to previous studies on common datasets. Hence, it is highly desirable to assess the performance of the transformer architecture for our task.

We propose a transformer architecture for peptide sequences that incorporates proven improvements to the original model. Specifically, we implement an adapted transformer model based on the vision transformer [13]. Previous studies have shown that changing the order of the so-called layer normalization in the architecture leads to better stability in training than the original version [17–19]. We will compare the performance of the proposed transformer model to state-of-the-art methods including Prosit [6], DeepDIA [8], and AutoRT [4] for regular, nonphosphorylated peptides. In addition, we will compare the transformer model with the same set of hyperparameters to two recent methods for phosphorylated peptides called DeepPhospho [16] and AlphaPeptDeep [20]. Note that we consider only the RT prediction component of the methods in the literature which also contain modules for MS/MS fragment intensity prediction or collisional cross section prediction.

Since the performance of a learning model largely depends on training data, we will train and validate the method using the same datasets as existing methods for benchmarking. We will also create an independent dataset to evaluate the accuracy and other design choices such as the constraint on maximal peptide length. Finally, we will also compare the performance of the adapted transformer architecture with the original one.

Significance of the study

We assess a deep learning model called transformer for retention time prediction. By training and evaluating the model on the same datasets as those used to develop previous methods, we show that the transformer outperforms currently used models. The user of the transformer model will also benefit from advanced software development and support from the AI community. The data and software code are made publicly available to facilitate further development of prediction tools for spectral library-based analysis of DIA-MS data.

2 | MATERIALS AND METHODS

2.1 | A transformer architecture for peptides

Figure 1 illustrates our transformer architecture for RT prediction. The last row in the figure is the input layer that encodes peptide sequences into tokens in a vector space. The encoder transforms the vector representation capturing the short range and long range dependency among amino acids in the input sequence. The learned representation of the first token is used as input for the prediction layer as in the BERT architecture [12].

2.1.1 | Amino acid embeddings

A transformer uses latent vectors of size d_{model} as input and output of each encoder block. We first convert each amino acid into a vector of size d_{model} through an embedding layer. Specifically, we use the Prosit annotation where a peptide sequence is a string of amino acids annotated by either 20 characters "ACDEFGHIKLMNPQRSTVWY" (unmodified) or 21 characters "ACDEFGHIKLMNPQRSTVWYo" where "o" denotes oxidation of methionine. Similarly, for phosphopeptides, we use the DeepPhospho annotation with a string of characters "ACDEFGHIKLMNPQRSTVWY1234*" where "1" denotes oxidation of methionine; "2", "3", and "4" denote phosphorylated serine, threonine, and tyrosine, respectively; and "*" denotes N-terminal acetylation. The input peptide sequence is converted to a vector of integers by replacing each amino acid of the sequence with a unique integer corresponding to its position in the string (i.e., "A" is 1, "C" is 2, "D" is 3 and so on). A class token CLS equal to the maximum value plus 1 (hence, either 21, 22, or 26) is inserted at the front of the obtained vector. The vector is subsequently mapped to a vector space using an embedding layer where the size of the vocabulary in the input data is CLS, the input length is the maximum sequence length, and the output dimension is d_{model}.

2.1.2 | Positional embeddings

The positional embeddings are added to the amino acid embeddings to retain positional information of the amino acid sequence. The result of the summation is the input to the encoder layers.

2.1.3 | Encoder

A transformer consists of N encoder blocks. Each block consists of two sub-blocks, a multi-head attention layer with a padding mask followed by a feed-forward network of size d_{ff}. Both sub-blocks have a residual connection [21] with a layer normalization [22] at the front and a dropout layer with a dropout rate r at the back. An architectural change from the original transformer (Figure S1) is that the normalization layer is placed prior to the multi-head attention layer and the feed-forward network layers, and inside the residual block.

2.1.4 | Multi-head attention

The multi-head attention layer allows the model to look at other positions in the input sequence to model short range and long range dependencies. It is a concatenation of h self-attention heads running in parallel. Multi-head attention allows the model to attend to information from different representation subspaces at different positions. Attention is a mathematical method to retrieve the value associated

16159861, 2023, 7-8, Downloaded from https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/pmic.202200041 by Readcube (Labtiva Inc.), Wiley Online Library on [27/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

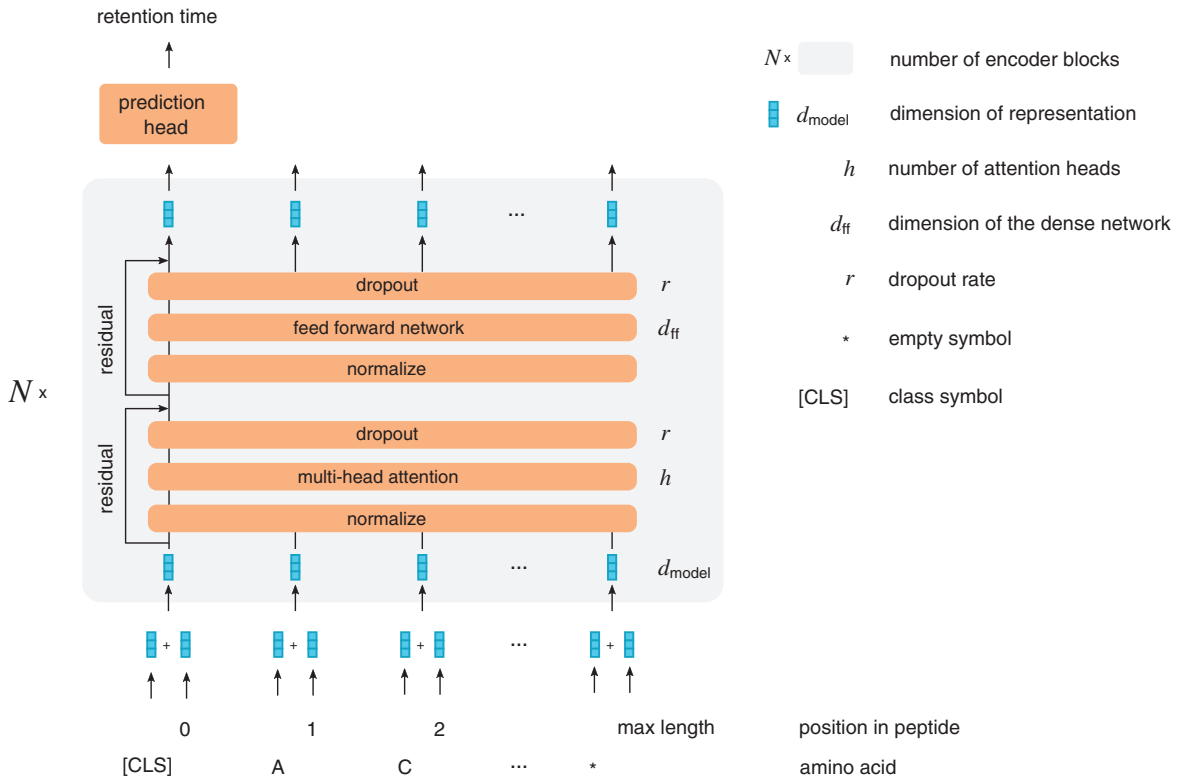

FIGURE 1 A transformer architecture for retention time prediction based on peptide sequences. The amino acids and their positions in the peptide are embedded in a d_{model} dimensional vector space and subsequently transformed through N layers of transformer encoding blocks. A virtual class token (CLS) is added to the front of each peptide and its learned representation is used as input to the prediction head. The number of layers N, the number of attention heads h, the dimension of the embedding space d_{model}, the dimension of the internal, fully connected network d_{ff}, and the dropout rate r are hyperparameters of the architecture.

with the key most similar to a given query. In the transformer model, the queries, keys, and values are the input embeddings for the first block and the output of the preceding encoder block for others. Like in other transformer models in the literature, we use d_{model}/h as the dimension of queries, keys, and values. Thus, the total computational cost is similar to that of single-head attention with full dimensionality d_{model}.

2.1.5 | Feed-forward network

The role of the feed-forward network is to process the output from the attention layer to better fit the input for the next attention layer. The feed-forward network consists of two dense layers. The first layer has a Gaussian error linear unit (GELU) activation [23] with d_{ff} outputs as in [12, 24]. The second layer transforms the data back to a d_{model} dimensional space.

2.1.6 | Dropout

We apply dropout [25] to the output of each sub-layer. In addition, we apply dropout to the sums of the input embeddings.

2.1.7 | Prediction head

We use two dense layers with a dropout layer in between. The first dense layer with d_{model} nodes and a GELU activation. Second dense layer outputs the RT prediction. This is similar to the last layer of the Prosit model.

2.2 | Implementation and training

We use the Tensorflow library (https://www.tensorflow.org/) for the transformer model. The source code of the encoder was adapted from the example at https://www.tensorflow.org/text/tutorials/transformer. Modifications include the placing of the normalization layer within the residual connection, positional embeddings replacing positional encodings, and the use of the GELU activation in the feed-forward layers.

We minimize the median absolute error (MAE) as the training objective function

$$\text{MAE} = \text{median}|\text{RT}_{\text{observed}} - \text{RT}_{\text{predicted}}|$$

16159861, 2023, 7-8, Downloaded from https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/pmic.202200041 by ReadCube (Labtiva Inc.), Wiley Online Library on [27/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

We use the Adam optimizer [26] with $\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 1e-9$ with a custom schedule. The learning rate at step s is

$$d_{model}^{-0.5} \cdot \min(s^{-0.5}, s \cdot w^{-1.5})$$

with the number of warm-up steps $w = 4000$ as in [10].

We perform random search for hyperparameter tuning using the TensorBoard tool (https://www.tensorflow.org/tensorboard). We train each model for 100 epochs with early stopping if the accuracy does not improve' for 20 epochs.

For the final model, we select the best model on the validation set after 2000 epochs.

The experiments were performed using a workstation with an Intel CPU i7-6850K at 3.60GHz, 128GB RAM main memory with an NVIDIA GeForce RTX 3090 card with 24GB GPU RAM, and a single NVIDIA A100 Tensor Core GPU with 40GB GPU RAM on the Dutch national supercomputer Snellius.

2.3 | Evaluation measure

We use the MAE to evaluate the performance of RT prediction models. An R script is used to calculate the evaluation metric from the text output of each model. The Wilcoxon signed-rank test is used to assess the statistical significance of the relative performance between models.

2.4 | Prosit model and data

The Python code is downloaded from https://github.com/kusterlab/prosit, the trained model from https://figshare.com/projects/prosit/35582, and the data from https://figshare.com/articles/dataset/ProteomeTools_-_Prosit_iRT_-_Data_update_missing_c_splitting/10043132. The experimental environment is Python 3.7, Tensorflow 2.2 and Keras 2.4.3. To test a new set of peptides, we convert the peptide sequences to Prosit numeric format and subsequently call the model evaluation module from within Python. The prediction result is written to a text file for further analysis in R. To train the transformer model, we use the exact training, validation, and holdout data associated with the publication.

2.5 | DeepDIA model and data

The source code for DeepDIA is downloaded from https://github.com/lmsac/DeepDIA. Due to the lack of processed data, we started from the raw mass spectrometry dataset PXD005573 [1]. We performed a peptide search of the set of 17 LC-MS/MS runs as used in the DeepDIA paper for the HeLa cells using MaxQuant version 1.6.14.0 with default settings [27]. The peptides in the MaxQuant msms.txt output are filtered for length from 7 to 50 amino acids. The median RT is used as a representative value for each peptide. The model takes a peptide sequence as input and converts it to a binary matrix, where each row represents an amino acid residue (20 rows in total), and each column stands for a residue position. The training, validation and holdout set are divided as specified in the DeepDIA publication. A third of the dataset is for holdout testing. The remaining two thirds are split again into two thirds for training and one third for validation. We subsequently used the published code to train the model. The experimental environment is Python 3.8, Tensorflow 2.2 and Keras 2.2.4. To test an external dataset, we adapted the DeepDIA Python code to accept a new input file and output to a text file for further processing in R.

2.6 | AutoRT model and data

We used the published version of AutoRT from https://github.com/bzhanglab/AutoRT in the following experimental environment: Python 3.7, Tensorflow 1.13.1, and Keras 2.2.4. We used the publicly available trained AutoRT model for evaluation. Like DeepDIA, each peptide was encoded into a binary matrix using one-hot encoding, in which each amino acid was represented as a binary vector of all zeros except for one entry set to one to indicate the category of the amino acid. To test a new set of peptides, we prepared the data format as required by AutoRT and subsequently executed a command line call to generate the prediction result in a text format. We used the published dataset, which was processed from the PXD006109 dataset [28], to train the transformer model. The AutoRT is an ensemble of 10 deep learning models. To train each model in the ensemble, the AutoRT algorithm splits the training data randomly into training and validation. Hence, the training and validation data for each model are different. We extracted the first split to train the transformer model. The holdout dataset is identical for the AutoRT model and the transformer model.

2.7 | An independent test set

We selected from the ProteomeXchange dataset PXD019038 [29] an LC-MS/MS run with the highest number of peptide identifications. Here the raw data were searched using MaxQuant 1.5.4.1 with default settings. The median RT is used as a representative value for each peptide. We filtered the peptides for length and oxidation modification appropriately for each prediction model.

2.8 | Phosphoproteomics predictive models and data

We considered two models for phosphopeptide RT prediction, namely, DeepPhospho (https://github.com/weizhenFrank/DeepPhospho) and AlphaPeptDeep (https://github.com/MannLabs/alphapeptdeep). The Python environments were prepared as in the software specification of each software package.

Four datasets used to develop DeepPhospho were prepared for model training and validation. First, the human phosphopeptide RT

16159861, 2023, 7-8, Downloaded from https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/pmic.202200041 by Readcube (Labtiva Inc.), Wiley Online Library on [27/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 Characteristics of three datasets used by the three corresponding state-of-the-art methods for retention time prediction.

Dataset	# train	# val.	# holdout	Max length	Mod.	# vocab.
Prosit	921,296	263,226	131,610	30	M(ox)	21
AutoRT	110,799	12,312	13,680	48	M(ox)	21
DeepDIA	29,714	14,858	22,286	50	-	20

M(ox) indicates the inclusion of oxidation of methionine modification.

data were downloaded from the supplementary data of [30] (removing phosphopeptides with the phosphosite Ascore ≤13 or RT outside of the range −5 to 115) and converted to both DeepPhospho and AlphaPeptDeep format. The RT was scaled to the range [0, 1]. Second, the mouse brain data MaxQuant search output file msms.txt was downloaded from PRIDE with the identifier PXD006637 [31]. Third, 24 Vero E6 raw DIA data PXD019113 [32] were processed using Spectronaut version 17.1 [33] using the *Chlorocebus sabaeus* reference proteome fasta file available in the repository (19,136 protein sequences). We used the same Spectronaut setting as DeepPhospho: Phospho (STY) as variable modifications, Best N Fragments per Peptide Max set to 25, PTM Min Localization Threshold set to 0.75, and Fragment ions m/z set to 200–2000. Fourth, 70 yeast raw DIA data PXD013453 [34] were searched using Uniprot *Saccharomyces cerevisiae* reference proteome (6060 protein sequences, downloaded 2023/01). The DeepPhospho Python function generate_dataset.py was used to generate peptide and RT pairs from MaxQuant search results and Spectronaut libraries after being exported to a plain text format. Both DeepPhospho and the transformer model work with the prepared datasets directly. For AlphaPeptDeep, we developed a Python script to further convert the data to the required format.

3 | RESULTS

3.1 | The transformer architecture is competitive to state-of-the-art architectures for RT prediction

The performance of a machine learning model is dependent on datasets. Hence, to properly assess a transformer model, we will use the same datasets that were used to train models currently applied in the field. Specifically, we will experiment with three deep learning models Prosit, AutoRT, and DeepDIA, which employ different variations of the LSTM architecture. The Prosit model consists of a bidirectional recurrent neural network with gated recurrent memory units (GRU) [35], a recurrent GRU layer and an attention layer [36], all with dropout layers at the front and the back. DeepDIA uses a hybrid architecture with a convolutional neural network (CNN) [37] and a bidirectional LSTM network [11]. AutoRT uses an ensemble of deep learning models with CNNs combined with bidirectional GRU networks. Note that DeepLC [9] is a more recent method, but its performance is equivalent to previous ones on unmodified peptides that we consider here.

The Prosit and AutoRT datasets were downloaded from publicly available resources while the DeepDIA dataset was obtained by reprocessing the raw data (Materials and methods). Table 1 summarizes the difference in training data size, maximal peptide length, and modifications accommodated by the three models. The Prosit model supports a maximal peptide length of 30 amino acids and oxidation of methionine modification. For DeepDIA, the maximal length is 50 and only unmodified peptides are considered. The AutoRT model supports peptides of length up to 48 and oxidation of methionine modification. The training dataset is used by the algorithm to optimize the model. The validation set is used to monitor the performance of the algorithm to determine the stopping criteria. The validation set is also used to determine hyperparameters of the model, and hence it is considered part of the training process. The holdout dataset is used for independent assessment. The Prosit dataset has the largest number of samples. However, there are duplicates of sequences with different RT values in the dataset.

Using the training and validation sets, we developed a single transformer model for all three datasets. Figure 1 depicts the architecture, which is based on the BERT architecture with a class token (CLS) at the beginning whose learned representation is used in the last layer for prediction. The model differs from the original transformer in the placing of the normalization layer and the residual connection, and the GELU activation (Figure S1). The RT values are normalized to [0,1] and a sigmoid activation with two fully connected layers is used as the prediction head. We implement a single Python source code for training, testing, and tuning in the Tensorflow environment. We perform three rounds of random search for hyperparameter optimization (Figure 2A), leading to a single parameter set for all three models, $N = 10$, $d_{model} = 256$, $d_{ff} = 1024$, $n_h = 8$, $d_r = 0.1$, and a batch size of 256.

The training and validation errors as functions of epoch are typical whereby minimizing training error drives down validation error (Figure S2). The performance on the validation sets does not improve noticeably after about 1000 epochs. And overfitting does not seem to occur at 2000 epochs. Figure 2B shows a scatter plot of the prediction against ground truth values on the holdout dataset of the model trained on the AutoRT dataset. The correlation between predicted and experimental RT values is very high ($R^2 = 0.997$). The MAE is 0.37, which is better than the result reported by the AutoRT study for other methods on this dataset, AutoRT (0.47), Prosit (0.72), GuanMCP2019 (0.57) [38], DeepMass (0.66) [7], and GPTime (2.37) [39]. Here all units are in minutes in the original RT space of the dataset. Among the aforementioned methods, all are deep learning-based except for GPTime. The results for the transformer model trained on the Prosit holdout data (MAE = 0.91) and the DeepDIA holdout data (MAE = 2.12) are shown in Figure S3.

16159861, 2023, 7-8, Downloaded from https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/pmic.202200041 by Readcube (Labtiva Inc.), Wiley Online Library on [27/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

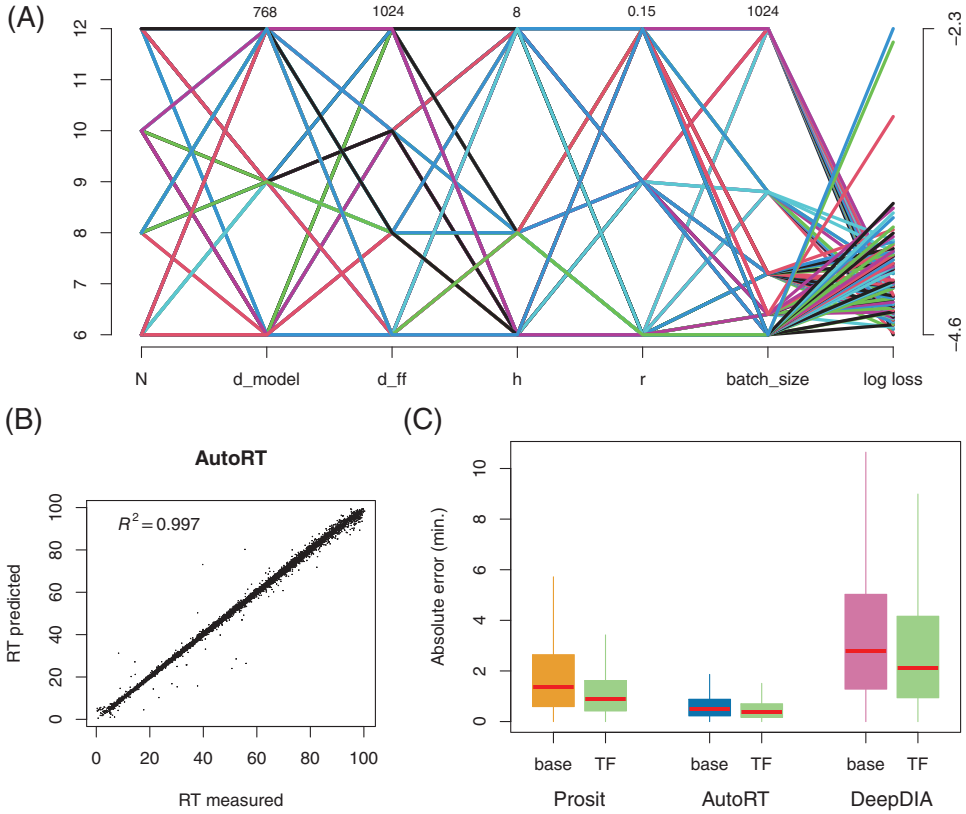

FIGURE 2 Training and evaluation of the transformer architecture. (A) Random search is performed for hyperparameter tuning. Each line represents a set of hyperparameter values. Each model is trained for 30 epochs and the performance on the validation set is recorded. (B) The correlation between the experimentally measured retention time (RT) and the predicted RT of the transformer model trained on the AutoRT dataset evaluated on the corresponding holdout dataset (10,000 random peptides are plotted). (C) The performance of each transformer model (TF) is compared to that of the corresponding base model. The boxplots show the medians in red, the range and the interquartiles of the absolute errors of all peptides on the holdout sets.

Figure 2C shows the boxplots of the absolute prediction errors on the holdout datasets of the three models. The transformer models were trained on the same training and validation set as the base model for Prosit and DeepDIA. For AutoRT, the dataset for training the first model in the ensemble was used. The MAEs (red lines), the interquartile ranges, and the error ranges of the transformer models are better than those of the corresponding base models (Wilcoxon signed-rank test p-values $< 1e{-}5$).

3.2 | Performance on an independent dataset

To assess the performance of the models on an independent dataset, we selected from a recently published dataset [29] an LC-MS/MS run with the highest number of peptides to avoid problems with cross-run normalization. For each model trained on Prosit data, DeepDIA data, and AutoRT data, we filtered the peptides appropriately for modification and maximum length. Table 2 summarizes the resulting datasets. Out of 15,322 peptides, the large majority are supported by the three models Prosit (14,667; 96%), AutoRT (14,790; 97%), and DeepDIA (14,706; 96%). The increase in peptide length does not substantially increase the number of peptides covered.

TABLE 2 The result of filtering according to the three models on an independent test set of 15322 peptides.

Filtering model	# remaining peptides	# in training	# not in training
Prosit	14,667	6852	7815
AutoRT	14,790	5255	9535
DeepDIA	14,706	7765	6941

Next, we separated peptides present in the training data of each model to have an accurate estimation of generalization error. Here the training data consists of both the training set and the validation set, and not the holdout set. As can be seen in Table 2, the number of peptides in the independent test set already present in training are similar: Prosit (47%), AutoRT (36%), and DeepDIA (53%). The numbers of remaining peptides are substantial for all three cases: Prosit (7815 peptides), AutoRT (9535 peptides), and DeepDIA (6941 peptides). The distribution of the peptide lengths shows a similar pattern across three training datasets (Figure S4).

The accuracy of each model is calculated using peptides not present in training. We use the predicted RT values of the peptides present in the training data to scale the result with a linear mapping with a

16159861, 2023, 7-8, Downloaded from https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/pmic.202200041 by Readcube (Labtiva Inc.), Wiley Online Library on [27/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

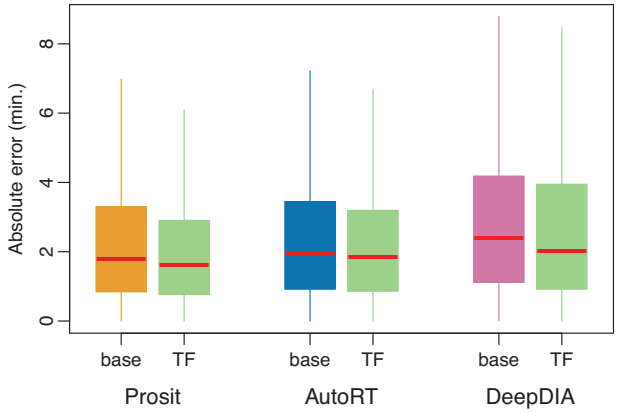

FIGURE 3 Boxplots of the absolute errors for the three transformer (TF) models and the corresponding base models on an independent test set.

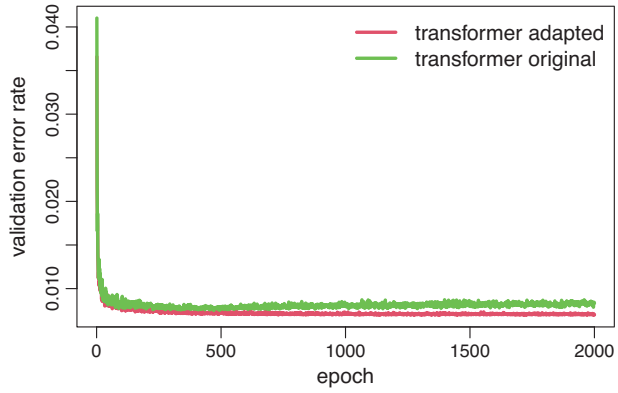

FIGURE 4 The validation error as a function of epoch for the adapted transformer architecture and the original architecture on the Prosit dataset.

least square estimation. Figure 3 shows the performance of the three transformer models and three existing models. The transformer outperforms all three existing deep learning models in terms of the median error rate, the interquartile range, and the error range (Wilcoxon signed-rank test p-values $< 1e{-}5$). The error rate of models based on the DeepDIA dataset is quite high, indicating that the number of training samples is insufficient (Table 1). Finally, the model trained on the Prosit dataset is the best of the three transformer models considered.

3.3 | Assessment of the adapted transformer architecture

We have adopted a recent variant of the transformer architecture which has several changes with respect to the original version. To assess their relative performance for RT prediction, we trained a model with the original architecture. Briefly, instead of positional embeddings, we used the same sine and cosine functions of different frequencies for the positional encodings [10]. The layer normalization is performed after the residual connection (Figure S1). Finally, the ReLU activation function is used in place of the GELU activation.

Figure 4 shows the performance on the validation set of the adapted transformer and the original transformer as a function of the training epoch for the Prosit dataset. The adapted transformer clearly outperforms the original transformer in terms of error rate. The stability of the adapted version is also better as the difference between consecutive epochs is much smaller than that of the original transformer. The same result is observed for the AutoRT dataset and the DeepDIA dataset (Figure S5).

3.4 | Assessment of the transformer architecture for phosphopeptides

Recent methods support RT prediction for phosphoproteomics data. DeepPhospho uses an LSTM block in front of transformer layers. AlphaPeptDeep is an environment to enable the user to easily create deep learning models for peptides. We will experiment with the published AlphaPeptDeep model consisting of CNN and LSTM.

We used four datasets used to develop DeepPhospho for evaluation (Materials and methods). The largest dataset [30] is converted to DeepPhospho format. Briefly, each peptide starts with either a character "*" for N-terminal acetylation or a character "@" for no N-terminal acetylation. Methionine oxidation is coded by character "1" and phosphorylation on STY is coded by "2", "3", and "4", respectively. In total, there are 2,04,331 in this set with peptide length ranges from 7 to 60 amino acids. The dataset is randomly split into training, validation, and holdout with ratio 8:1:1. The training set and the validation set are used to train and monitor the transformer and DeepPhospho. The two sets are combined to train AlphaPeptDeep which has a single dataset as input. The holdout is used for validation for all models.

We used the same transformer architecture developed in 3.1 for nonphosphorylated peptide without any further fine tuning. Thus, the hyperparameters are $N = 10$, $d_{model} = 256$, $d_{ff} = 1024$, $n_h = 8$, $d_r = 0.1$, and a batch size of 256. For DeepPhospho, we use the provided configuration to train an ensemble of five deep learning models with the number of layers ranging from 4 to 8. For AlphaPeptDeep, we trained the published model for 2000 epochs.

Figure 5A shows the boxplots of the absolute prediction errors on the holdout datasets of the three models. The transformer outperforms AlphaPeptDeep and DeepPhospho in terms of the MAEs (red lines; transformer MAE = 1.34 min; DeepPhospho MAE = 1.93 min; AlphaPeptDeep MAE = 1.61 min), the interquartile ranges, and the error ranges (all Wilcoxon signed-rank test p-values $< 1e{-}5$). Analysis of the transformer model predictions shows no correlation between peptide lengths and accuracies. However, there is a strong correlation between the number of modifications on a peptide and the accuracy of its RT prediction (Figure S6).

Table 3 summarizes the three datasets used as external validation. The number of phosphopeptides in PXD006637 is similar to that

16159861, 2023, 7-8, Downloaded from https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/pmic.202200041 by Readcube (Labtiva Inc.), Wiley Online Library on [27/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

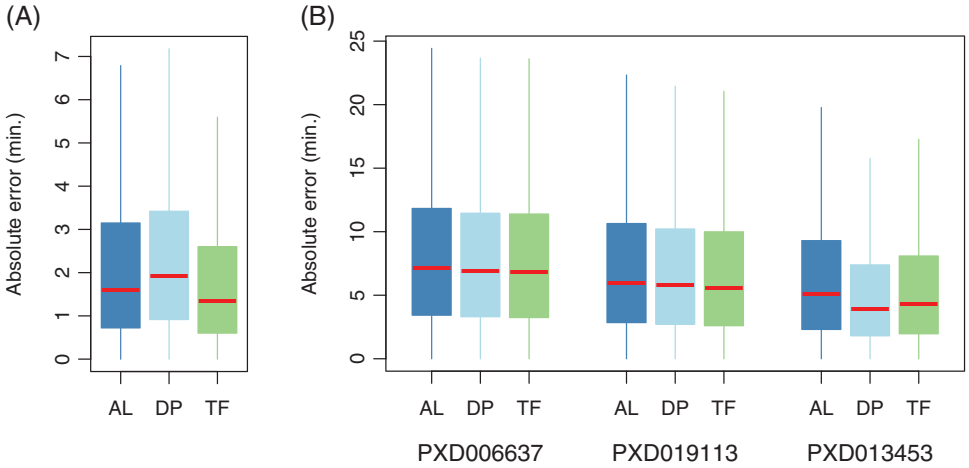

FIGURE 5 Performance of the transformer architecture (TF) versus AlphaPeptDeep (AL) and DeepPhospho (DP). The boxplots show the medians in red, the range and the interquartiles of the absolute errors of all peptides. (A) The result on the holdout test set. (B) The results on the three independent datasets (Note that these datasets were used in the development of DeepPhospho).

TABLE 3 Datasets for independent evaluation of the deep learning models for phosphoproteomics.

Datasets	Description	# peptides	# in training	# not in training	Min length	Max length
PXD006637	Mouse DDA	57,681	8000	49,681	7	51
PXD019113	VeroE6 DIA	24,050	9762	14,288	7	49
PXD013453	Yeast DIA	61,607	18	61,589	7	52

reported by DeepPhospho, whereas the number of phosphopeptides in PXD019113 (24,050) and PXD013453 (61,607) are quite different from those reported in DeepPhospho, 43,405 and 35,808, respectively. A possible explanation for this discrepancy is the version of Spectronaut we used is more recent, version 17.1, in contrast to version 14.5 used in DeepPhospho. Phosphopeptide search for DIA-MS data have been drastically updated in recent years.

To assess the performance on the three external datasets, the training set and the validation set is combined to create a list of peptides used in developing the prediction model. The accuracy of each model is calculated based on peptides which were not present in this list. Note that only 18 peptides overlap between the yeast dataset PXD013453 and the training data derived from human phosphoproteome. This is likely due to the difference of the proteomes. Our analysis shows that there are less than 500 tryptic peptide sequences with length ≥7 in common between the set of more than 5,38,000 tryptic peptide sequences from the canonical human proteome and the set of more than 1,41,000 tryptic peptide sequences from the canonical yeast proteome.

The predicted RT values of the peptides present in the training data is used to scale the result with a linear mapping with a least square estimation as in 3.2. Figure 5B shows that the transformer model outperforms AlphaPeptDeep in all three datasets (Wilcoxon signed-rank test p-values $< 1e{-}5$), and it is better than DeepPhospho on the PXD019113 but worse on the PXD013453 (Wilcoxon signed-rank test p-values $< 1e{-}5$). On the PXD006637, the MAE of the transformer is slightly better than that of DeepPhospho (6.81 min vs. 6.89 min), but the statistical test favors the DeepPhospho model.

Finally, we visualized the amino acid embeddings of the transformer models using principal component analysis (Figure S7). We observed that the modified amino acids are considerably different from the unmodified ones for both nonphosphorylated dataset (oxidation of methionine) and phosphoproteomics dataset (all modifications covered).

4 | DISCUSSION

We have implemented a transformer architecture that outperforms currently used state-of-the-art methods for RT prediction. The experimental result shows that the training data are important, and in our study the Prosit dataset delivered the best result for nonphosphorylated peptides. In addition, the transformer model without any additional hyperparameter tuning delivers the best predictive result for phosphoproteomics datasets in comparison to DeepPhospho and AlphaPeptDeep.

The variant of the transformer we implemented is better than the original transformer model, thereby confirming previous observations in the literature for other types of data [17]. We expect that the change of the order of the layer normalization and multi-head attention contribute most to the improvement. There is some theoretical

16159861, 2023, 7-8, Downloaded from https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/pmic.202200041 by Readcube (Labtiva Inc.), Wiley Online Library on [27/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

explanation on why pre-normalization is better [17–19]. In the original work of transformer by Vaswani et al. [10], the authors experimented with positional encoding and positional embedding and concluded that there was no change in performance. Here, we have made some design choices based on our preference from the existing BERT and vision transformer architecture. We leave further analysis and fine-tuning of the transformer architecture for future research.

Unlike other deep learning models with elaborate architectures, our model for RT prediction is highly compact. We only add an embedding layer for input and an output layer for prediction. Hence, existing software for the transformer model can be readily employed. A high-performance transformer for proteomics is interesting because the architecture is highly successful in various other application fields and therefore widely supported by the community. The transformer architecture already has software robustness, scalability, and ease of deployment. As accuracy of deep learning models for RT prediction for unmodified peptides is reaching a saturation point as illuminated here by the overlapping boxplots in Figure 3 and in recent publications [4, 9], other factors such as usability will become important to facilitate a wider user base. This is particularly important for users with limited experience with machine learning tools.

Note that four out of five methods in the literature we considered have modules for MS/MS ion intensity prediction and are integrated in end-to-end systems for DIA-MS data analysis. In our opinion, MS/MS intensity prediction can be treated separately because of the different underlying physical properties. Our results and datasets will enable either fine-tuning these end-to-end systems or replacing their RT predictive component.

A common benchmarking environment is important to further develop prediction tools for proteomics data analysis [40]. Our effort in deploying the five existing deep learning models was considerable. So was our effort in data preparation. The availability of our software and datasets is a contribution in this regard.

AUTHOR CONTRIBUTION

Thang V. Pham, Vinh V. Nguyen, and Connie R. Jimenez conceived the study. Thang V. Pham, Vinh V. Nguyen, and Duong Vu developed the method and experimental software. Thang V. Pham, Vinh V. Nguyen, Alex A. Henneman and Robin A. Richardson reviewed the software, prepared the data, and performed experiments. Thang V. Pham, Duong Vu, Alex A. Henneman, Robin A. Richardson, Sander R. Piersma, and Connie R. Jimenez interpreted the results and wrote the manuscript.

ACKNOWLEDGEMENTS

This work was supported by the Netherlands eScience Center, grant no. ASDI.2020.014. This work made use of the Dutch national e-infrastructure with the support of the SURF Cooperative using grant no. EINF-3550.

CONFLICT OF INTEREST STATEMENT

The authors declare no conflict of interest.

DATA AVAILABILITY STATEMENT

Software codes and data files are available on GitHub https://github.com/aiproteomics/rt.

REFERENCES

1. Bruderer, R., Bernhardt, O. M., Gandhi, T., Xuan, Y., Sondermann, J., Schmidt, M., Gomez-Varela, D., & Reiter, L. (2017). Optimization of experimental parameters in data-independent mass spectrometry significantly increases depth and reproducibility of results. *Molecular & Cellular Proteomics*, *16*(12), 2296–2309.
2. Demichev, V., Messner, C. B., Vernardis, S. I., Lilley, K. S., & Ralser, M. (2020). DIA-NN: neural networks and interference correction enable deep proteome coverage in high throughput. *Nature Methods*, *17*(1), 41–44.
3. Sinitcyn, P., Hamzeiy, H., Salinas Soto, F., Itzhak, D., McCarthy, F., Wichmann, C., Steger, M., Ohmayer, U., Distler, U., Kaspar-Schoenefeld, S., Prianichnikov, N., Yılmaz, Ş., Rudolph, J. D., Tenzer, S., Perez-Riverol, Y., Nagaraj, N., Humphrey, S. J., & Cox, J. (2021). MaxDIA enables library-based and library-free data-independent acquisition proteomics. *Nature Biotechnology*, *39*(12), 1563–1573.
4. Wen, B., Li, K., Zhang, Y., & Zhang, B. (2020). Cancer neoantigen prioritization through sensitive and reliable proteogenomics analysis. *Nature Communications*, *11*(1), 1759.
5. Dorfer, V., Maltsev, S., Winkler, S., & Mechtler, K. (2018). CharmeRT: Boosting peptide identifications by chimeric spectra identification and retention time prediction. *Journal of Proteome Research*, *17*(8), 2581–2589.
6. Gessulat, S., Schmidt, T., Zolg, D. P., Samaras, P., Schnatbaum, K., Zerweck, J., Knaute, T., Rechenberger, J., Delanghe, B., Huhmer, A., Reimer, U., Ehrlich, H. C., Aiche, S., Kuster, B., & Wilhelm, M. (2019). Prosit: Proteome-wide prediction of peptide tandem mass spectra by deep learning. *Nature Methods*, *16*(6), 509–518.
7. Tiwary, S., Levy, R., Gutenbrunner, P., Salinas Soto, F., Palaniappan, K. K., Deming, L., Berndl, M., Brant, A., Cimermancic, P., & Cox, J. (2019). High-quality MS/MS spectrum prediction for data-dependent and data-independent acquisition data analysis. *Nature Methods*, *16*(6), 519–525.
8. Yang, Y., Liu, X., Shen, C., Lin, Y., Yang, P., & Qiao, L. (2020). In silico spectral libraries by deep learning facilitate data-independent acquisition proteomics. *Nature Communications*, *11*(1), 146.
9. Bouwmeester, R., Gabriels, R., Hulstaert, N., Martens, L., & Degroeve, S. (2021). DeepLC can predict retention times for peptides that carry as-yet unseen modifications. *Nature Methods*, *18*(11), 1363–1369.
10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, p. 30.
11. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.
12. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Volume 1 (Long and Short Papers). https://doi.org/10.48550/arXiv.1810.04805
13. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *Proceedings of the International Conference on Learning Representations 2021*. https://doi.org/10.48550/arXiv.2010.11929
14. Rives, A., Meier, J., Sercu, T., Goyal, S., Lin, Z., Liu, J., Ma, J., & Fergus, R. (2021). Biological structure and function emerge from scaling unsu-

16159861, 2023, 7-8, Downloaded from https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/pmic.202200041 by Readcube (Labtiva Inc.), Wiley Online Library on [27/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

pervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, *118*(15), e2016239118.
15. Ekvall, M., Truong, P., Gabriel, W., Wilhelm, M., & Käll, L. (2022). Prosit Transformer: A transformer for prediction of MS2 spectrum intensities. *Journal of Proteome Research*, *21*(5), 1359–1364.
16. Lou, R., Liu, W., Li, R., Li, S., He, X., & Shui, W. (2021). DeepPhospho accelerates DIA phosphoproteome profiling through in silico library generation. *Nature Communications*, *12*(1), 6685.
17. Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T. -Y. (2020). On layer normalization in the transformer architecture. Paper presented at the International Conference on Machine Learning.
18. Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*. https://doi.org/10.48550/arXiv.1904.10509
19. Wang, Q., Li, B., Xiao, T., Zhu, J., Li, C., Wong, D. F., & Chao, L. S. (2019). Learning deep transformer models for machine translation. *Proceedings of the Annual Meeting of the Association for Computational Linguistics 2019 arXiv:1906.01787*. https://doi.org/10.48550/arXiv.1906.01787
20. Zeng, W. -F., Zhou, X. -X., Willems, S., Ammar, C., Wahle, M., Bludau, I., Voytik, E., Strauss, M. T., & Mann, M. (2022). AlphaPeptDeep: A modular deep learning framework to predict peptide properties for proteomics. *Nature Communications*, *13*(1), 7238.
21. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Paper presented at the Proceedings of the IEEE conference on computer vision and pattern recognition.
22. Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*. https://doi.org/10.48550/arXiv.1607.06450
23. Hendrycks, D., & Gimpel, K. (2016). Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*. https://doi.org/10.48550/arXiv.1606.08415
24. Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., & Clark, J. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*, 1877–1901.
25. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, *15*(1), 1929–1958.
26. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *Proceedings of the International Conference on Learning Representations 2015*. https://doi.org/10.48550/arXiv.1412.6980
27. Cox, J., & Mann, M. (2008). MaxQuant enables high peptide identification rates, individualized ppb-range mass accuracies and proteome-wide protein quantification. *Nature Biotechnology*, *26*(12), 1367–1372.
28. Meier, F., Geyer, P. E., Virreira Winter, S., Cox, J., & Mann, M. (2018). BoxCar acquisition method enables single-shot proteomics at a depth of 10,000 proteins in 100 minutes. *Nature Methods*, *15*, 440–448.
29. Van Linde, M. E., Labots, M., Brahm, C. G., Hovinga, K. E., De Witt Hamer, P. C., Honeywell, R. J., de Goeij-de Haas, R., Henneman, A. A., Knol, J. C., Peters, G. J., Dekker, H., Piersma, S. R., Pham, T. V., Vandertop, W. P., Jiménez, C. R., & Verheul, H M W. (2022). Tumor drug concentration and phosphoproteomic profiles after two weeks of treatment with sunitinib in patients with newly diagnosed glioblastoma. *Clinical Cancer Research*, *28*(8), 1595–1602.
30. Lawrence, R. T., Searle, B. C., Llovet, A., & Villén, J. (2016). Plug-and-play analysis of the human phosphoproteome by targeted high-resolution mass spectrometry. *Nature Methods*, *13*(5), 431–434.
31. Liu, J. J., Sharma, K., Zangrandi, L., Chen, C., Humphrey, S. J., Chiu, Y. -T., Spetea, M., Liu-Chen, L. Y., Schwarzer, C., & Mann, M. (2018). In vivo brain GPCR signaling elucidated by phosphoproteomics. *Science*, *360*(6395), eaao4927.
32. Bouhaddou, M., Memon, D., Meyer, B., White, K. M., Rezelj, V. V., Marrero, M. C., Polacco, B. J., Melnyk, J. E., Ulferts, S., Kaake, R. M., Batra, J., Richards, A. L., Stevenson, E., Gordon, D. E., Rojc, A., Obernier, K., Fabius, J. M., Soucheray, M., Miorin, L., & Moreno, E. (2020). The global phosphorylation landscape of SARS-CoV-2 infection. *Cell*, *182*(3), 685–712.e619.
33. Bruderer, R., Bernhardt, O. M., Gandhi, T., Miladinović, S. M., Cheng, L. -Y., Messner, S., Ehrenberger, T., Zanotelli, V., Butscheid, Y., Escher, C., Vitek, O., Rinner, O., & Reiter, L. (2015). Extending the limits of quantitative proteome profiling with data-independent acquisition and application to acetaminophen-treated three-dimensional liver microtissues. *Molecular & Cellular Proteomics*, *14*(5), 1400–1410.
34. Leutert, M., Rodríguez-Mias, R. A., Fukuda, N. K., & Villén, J. (2019). R2-P2 rapid-robotic phosphoproteomics enables multidimensional cell signaling studies. *Molecular Systems Biology*, *15*(12), e9021.
35. Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *Proceedings of the NIPS 2014 Deep Learning and Representation Learning Workshop*. https://doi.org/10.48550/arXiv.1412.3555
36. Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *Proceedings of the International Conference on Learning Representations 2015*. https://doi.org/10.48550/arXiv.1409.0473
37. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436–444.
38. Guan, S., Moran, M. F., & Ma, B. (2019). Prediction of LC-MS/MS properties of peptides from sequence by deep learning. *Molecular & Cellular Proteomics*, *18*(10), 2099–2107.
39. Maboudi Afkham, H., Qiu, X., The, M., & Käll, L. (2017). Uncertainty estimation of predictions of peptides' chromatographic retention times in shotgun proteomics. *Bioinformatics*, *33*(4), 508–513.
40. Shouman, O., Gabriel, W., Giurcoiu, V. - G., Sternlicht, V., & Wilhelm, M. (2022). PROSPECT: Labeled tandem mass spectrometry dataset for machine learning in proteomics. Paper presented at the Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track.

SUPPORTING INFORMATION

Additional supporting information may be found online https://doi.org/10.1002/pmic.202200041 in the Supporting Information section at the end of the article.

How to cite this article: Pham, T. V., Nguyen, V. V., Vu, D., Henneman, A. A., Richardson, R. A., Piersma, S. R., & Jimenez, C. R. (2023). A transformer architecture for retention time prediction in liquid chromatography mass spectrometry-based proteomics. *Proteomics*, *23*, e2200041. https://doi.org/10.1002/pmic.202200041

16159861, 2023, 7-8, Downloaded from https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/pmic.202200041 by Readcube (Labtiva Inc.), Wiley Online Library on [27/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License